

Master's Thesis Topic Proposal

Domain: Machine Learning & Image Processing | Application area: Medical Imaging | Prepared for guide review and approval

The four topics below are all in the medical imaging domain and are grouped into two Image Processing topics and two Machine Learning topics. Each entry states what already exists in current research/tools, what new contribution this project will add, the technology stack to be used, and how the final web application will function. Any one topic (or a combination of one Image Processing + one Machine Learning topic) can be selected as the final thesis project.

SECTION A · IMAGE PROCESSING TOPICS

TOPIC 1

Enhancement and Noise Reduction of Low-Quality Medical X-Ray Images

What already exists

Classical enhancement methods (histogram equalization, CLAHE, wavelet-based denoising) are well established, and some deep learning denoising autoencoders exist. Most existing work stops at showing a visually improved image, without measuring whether the improvement actually helps a diagnosis system.

What we will add (novelty)

- Implement and compare 3–4 enhancement methods (CLAHE, wavelet denoising, deep denoising autoencoder, GAN-based enhancer) on the same noisy X-ray dataset.
- Evaluate objectively using image quality metrics: PSNR and SSIM, instead of just visual comparison.
- Task-driven evaluation (main contribution): pass enhanced images through a pneumonia classifier and show measurable improvement in diagnosis accuracy — connecting image processing quality to a real clinical outcome, not just image aesthetics.

Technology to be used

Python, OpenCV, scikit-image, TensorFlow/Keras (for the autoencoder/GAN), NumPy, Matplotlib for result visualization.

How the website will work

A web page where a user uploads a noisy/low-quality X-ray, selects (or auto-applies) an enhancement method, and views a side-by-side before/after comparison along with the PSNR/SSIM score of the improvement, built using Flask (Python backend) with an HTML/CSS interface.

TOPIC 2

Automated Segmentation of Lung Regions in Chest X-Rays for Diagnostic Preprocessing

What already exists

U-Net based lung segmentation is a known technique in medical image processing research, with several public implementations achieving good Dice/IoU scores. However, segmentation is usually treated as a standalone task, rarely tested for its actual effect on downstream diagnosis quality.

What we will add (novelty)

- Train a U-Net (or a lightweight variant) to automatically isolate the lung region from the full chest X-ray, removing background, ribcage clutter, and text artifacts.
- Main contribution: show that classifying only the segmented lung region improves downstream classifier accuracy compared to classifying the raw, unsegmented image.
- Build a lightweight/faster segmentation model suited for real-time use in a deployed web app, rather than a heavy research-only model.

Technology to be used

Python, TensorFlow/Keras (U-Net), OpenCV for pre/post-processing, NumPy, scikit-learn for evaluation metrics (Dice score, IoU).

How the website will work

A web page where a user uploads a chest X-ray; the app displays the original image alongside the automatically segmented lung mask overlay, with the segmented region available to feed into a diagnosis step — built with Flask backend and an HTML/CSS/JavaScript frontend.

SECTION B · MACHINE LEARNING TOPICS

TOPIC 3

Explainable Pneumonia Detection from Chest X-Rays using Deep Learning

What already exists

CNN-based pneumonia classifiers (using transfer learning models like DenseNet121, ResNet50) already achieve high accuracy on the public Kaggle chest X-ray dataset. However, most function as black boxes, giving a prediction without showing which part of the image influenced the decision — a well-documented trust barrier in medical AI adoption.

What we will add (novelty)

- Grad-CAM explainability: generate a visual heatmap for every prediction, showing exactly which lung region the model focused on — directly addressing the black-box criticism.
- Generalization test: train on one X-ray dataset and test on a different dataset/hospital source to check whether the model's accuracy holds up outside its training data — a known open problem in medical AI research.
- Confidence-aware output: flag uncertain predictions (e.g., 40–60% confidence) as 'needs radiologist review' instead of forcing a binary answer, closer to how such a tool would be used in practice.

Technology to be used

Python, TensorFlow/Keras (DenseNet121 transfer learning), Grad-CAM implementation, OpenCV for heatmap overlay, scikit-learn for evaluation (accuracy, precision, recall, F1, AUC-ROC), Flask for deployment.

How the website will work

A web page where a user uploads a chest X-ray image; the app returns a Normal/Pneumonia prediction with a confidence score, along with a Grad-CAM heatmap overlay showing the region of the lung that drove the decision — built with a Flask backend (handles upload, prediction, heatmap generation) and an HTML/CSS frontend for the upload form and result display.

TOPIC 4

Predictive Machine Learning Model for Multi-Class Chest Disease Classification

What already exists

Binary (Normal vs. Pneumonia) classifiers are common and well studied. Multi-class chest disease classification (e.g., Pneumonia, COVID-19, Tuberculosis, Normal) is less mature, and existing work rarely compares multiple architectures or properly addresses the severe class imbalance found in multi-disease datasets.

What we will add (novelty)

- Model comparison study: train and compare 3–4 architectures (ResNet50, EfficientNet, DenseNet121, a Vision Transformer) on the same multi-class dataset, analyzing accuracy vs. inference speed vs. model size — giving a full, genuinely useful results chapter.
- Class imbalance handling: implement and compare strategies (class weighting, focal loss, oversampling) since multi-class medical datasets are usually heavily imbalanced, and report which strategy performs best.
- Optional extension: add Grad-CAM explainability per class for consistency with Topic 3.

Technology to be used

Python, TensorFlow/Keras (multiple pretrained backbones), scikit-learn (class weighting, metrics), imbalanced-learn (oversampling techniques), Flask for deployment.

How the website will work

A web page where a user uploads a chest X-ray image and the app returns the predicted disease class out of multiple possible categories, along with per-class confidence scores displayed as a simple bar chart — built with a Flask backend and an HTML/CSS/JavaScript frontend.

Note: All datasets used are publicly available (Kaggle/open medical imaging repositories) at no cost. All proposed systems are academic/research demonstrations and are not intended for real clinical diagnostic use.